

TDA4j

The TDA4j library implements persistent homology and related techniques from computational and applied topology, in a library designed for ease of use, ease of access from Matlab and java-based systems, and ease of extensions for further research projects and approaches.

The library is based on previous libraries from the Computational Topology workgroup at Stanford University.

For persistent homology and its capabilities, we recommend the survey article [Topology and Data](#) by Gunnar Carlsson.

On this site we include a fully compiled [ScalaDoc](#) documentation.

Tutorials for TDA4j

TODO: We want to port Henry Adams' excellent JavaPlex tutorials.

Placeholder page; nothing ported yet. When per-language (Java/Scala) tabbed examples are needed here, Laika's equivalent of Paradox's language-group tabs is the `@:select/@:choice` directive pair, backed by a `laika.selections` block in `directory.conf` — not yet wired up since there's no real content to switch between.

User Guide for TDA4j

TDA4j implements persistent homology and related techniques from computational and applied topology. This guide assumes you know what a simplicial complex, a filtration, and a persistence barcode are — it does not assume you know Scala. If you want to understand *why* the library is built the way it is, or you're planning to write new code against it, see the [Developer's Guide](#) instead; this page is about getting things done as a caller.

Quick-start: Scala

Snippets included via `@:snip` (with a source-file link) are compiled and exercised directly by the test suite. The rest are illustrative and hand-maintained, not mechanically checked — if you find one has drifted, trust the source over this page.

Imports

TDA4j's package is split into subpackages (`algebra`, `cells`, `streams`, `homology`, `alpha`, ...). Bring in what you need with the `{given, *}` form — a plain `import pkg.*` does **not** bring `given` instances (coefficient fields, orderings) into scope in Scala 3:

```
import org.appliedtopology.tda4j.algebra.{given, *}
import org.appliedtopology.tda4j.cells.{given, *}
import org.appliedtopology.tda4j.streams.{given, *}
import org.appliedtopology.tda4j.homology.{given, *}
```

The rest of this guide assumes these four imports (plus `alpha.{given, *}` where `alpha` complexes come up).

Building and taking the boundary of a simplex

```
// Coefficients need an explicit Field instance in scope -- there is no default
// one for Double.
// DoubleApproximated treats two coefficients as equal within epsilon, which
// matters for the
// zero-checks that drive chain reduction.
given Double is Field = Field.DoubleApproximated(1e-9)

val triangle = Simplex(1, 2, 3)      // same as #(1, 2, 3)
triangle.boundary[Double]            // Seq((Simplex(2,3), 1.0), (Simplex(1,3),
// -1.0), (Simplex(1,2), 1.0))
```

A full Vietoris-Rips persistence computation

```
given Double is Field = Field.DoubleApproximated(1e-9)
given ctx: TDAContext[Int, Double, Double]()
import ctx.{*, given}

val points: Array[Array[Double]]
  = Array(Array(0.0, 0.0), Array(1.0, 0.0), Array(0.5, 0.8))
val metricSpace = EuclideanMetricSpace(points)
val stream = EnumeratingCofaceSimplexStream(metricSpace, maxFiltrationValue
  = Some(2.0))

val state = ctx.persistentHomology(stream)
state.barcodeAt(Double.PositiveInfinity).foreach(println)
```

`TDAContext[VertexT, CoefficientT, FiltrationT]` bundles the naive, reference-grade persistence engine together with chain-arithmetic operators, so `1.0 # #(1,2) - #(2,3)` works directly once `ctx`'s members are imported. It's a good default for exploration and for anything where you want to query the diagram at intermediate filtration values or get representative cycles back (`state.diagramAt(f)/state.barcodeAt(f)`) — see "Which persistence engine?" below for when a different engine is worth reaching for instead.

A default worth knowing: `EnumeratingCofaceSimplexStream` and the other Vietoris-Rips stream implementations default `maxFiltrationValue` to the point cloud's own *minimum enclosing radius*, not unbounded, since nothing past that radius contributes new homology. Pass `Some(Double.PositiveInfinity)` explicitly if you want the old always-unbounded behavior.

Alpha complexes

```
import org.appliedtopology.tda4j.alpha.{given, *}

val shape = AlphaShapes(points.toSeq, dispatch = "helix") // or "DQP"
```

`AlphaShapes(points)` with no `dispatch`, or `dispatch = "default"`, always resolves to "helix" — ask for "DQP" explicitly if you want it. See "Which alpha-complex backend?" below for the tradeoffs.

Cech complexes

```
val cechStream = CechCofaceSimplexStream(metricSpace, maxFiltrationValue
  = Some(2.0))
val homology = SimplicialHomologyContext[Int, Double, Double]
().persistentHomology(cechStream)
```

`maxFiltrationValue` here is a Cech **radius**, not a Vietoris-Rips diameter — the two aren't interchangeable units. Only the naive engine (`SimplicialHomologyContext/CellularHomologyContext`) is used for Cech complexes; the packed Ripser engine's optimizations don't carry over (see the [Developer's Guide](#)).

Sheehy's sparse/approximate Vietoris-Rips filtration

```
val sheehyStream = SheehyRipsSimplexStream(metricSpace, epsilon = 0.5)
val homology = SimplicialHomologyContext[Int, Double, Double]
().persistentHomology(sheehyStream)
```

A $(1+\epsilon)$ -multiplicative approximation to plain Vietoris-Rips's own barcode, built from a complex that's linear-sized (for point sets of bounded doubling dimension) rather than the full Vietoris-Rips complex — fewer simplices to reduce at the same scale range, at the cost of a controlled, quantified loss of precision (Cavanna, Jahanseir & Sheehy 2015). Smaller `epsilon` means less sparsification and a closer approximation to plain Vietoris-Rips; `epsilon` must be strictly between 0 and 1. Like Cech above, only `naive/chunks/cohomology` are used — the packed Ripser engine's optimizations assume a plain `max-pairwise-distance` filtration functional, which this construction's own sparsification and vertex "vanishing" don't satisfy (see the [Developer's Guide](#)).

Witness complexes

```
val ambient = EuclideanMetricSpace(points)
val landmarks = LandmarkSelector.maxmin(ambient, numLandmarks = 20).landmarks

val lazyStream = LazyWitnessSimplexStream(ambient, landmarks) // nu = 2,
// JavaPlex's own default
val homology = SimplicialHomologyContext[Int, Double, Double]
().persistentHomology(lazyStream)
```

De Silva & Carlsson's witness complex (2004; the construction behind most of the JavaPlex tutorials) builds a complex over a small **landmark** subset of your point cloud, using every point (landmarks included) as a **witness** — useful when the full point cloud is too large to build a Vietoris-Rips complex over directly. `LandmarkSelector.maxmin` (sequential furthest-point sampling, a covering-radius guarantee) and `LandmarkSelector.random` (seeded, cheaper, no coverage guarantee) both work over any `FiniteMetricSpace[Int]` — a point cloud or a precomputed distance matrix.

Every emitted `Simplex[Int]`'s vertices are **local landmark indices** (0 until `landmarks.size`), not indices into your original point cloud — map back through `landmarks(i)` yourself (`matlab.TDA4j` does this for you).

Two variants, matching JavaPlex's own two classes:

- `LazyWitnessSimplexStream` (JavaPlex's `LazyWitnessStream`) — a flag/clique complex, so it also works directly with the packed Ripser engine (`PackedRipserCohomologyContext`) by handing it a `WitnessMetricSpace` instead of a stream: `PackedRipserCohomologyContext(WitnessMetricSpace(WitnessGeometry(ambient, landmarks), nu = 2), maxDimension)`. The `nu` parameter (0, 1, or 2, default 2) controls how forgiving a witness's own threshold is — see `WitnessMetricSpace`'s own doc.
- `WitnessCofaceSimplexStream` (JavaPlex's plain `WitnessStream`) — NOT a flag complex, so `engine=ripser/` chunks don't apply; use the naive or cohomology engine instead. `maxFiltrationValue` here defaults to `+Infinity` (untruncated), not the point cloud's enclosing radius — the enclosing-radius shortcut is only valid for flag complexes.

```
val geometry = WitnessGeometry(ambient, landmarks)
val generalStream = WitnessCofaceSimplexStream(geometry, maxFiltrationValue
= 2.0)
```

Pick a finite `maxFiltrationValue` for the general variant, or cap dimension with `LimitedCofaceSimplexStream` — its default is `+Infinity`, and nothing prunes an unbounded enumeration, so a direct call can reach the full $2^{\text{landmarks.size}}$ power set. `matlab.TDA4j` always caps dimension for you. Prefer `witnessVariant=lazy` (the default) when the flag-complex behavior is acceptable — it unlocks `engine=ripser`, and at tutorial scale (1000 points, 50 landmarks, threshold $2R$ — R the landmark selection's own covering radius, the JavaPlex tutorial's own recommended threshold) that engine choice is where nearly all the speed difference actually is: `lazy+ripser` $\sim 0.2s$ vs. `lazy+naive` $\sim 11.2s$ (same complex, same bar count, engine alone) vs. `general+naive` $\sim 11.5s$ (variant alone, engine held to naive, indistinguishable from `lazy+naive` at single-trial resolution) — measured, not inferred (single-trial numbers on one machine; see `.claude/WORKLOG-witness-complex.md`).

The two-step recipe from MATLAB/CLI: select landmarks, read R , then compute

The one-shot `complex=witness path` (below, in "Calling from MATLAB or Java") picks landmarks internally and never reports back the covering radius R — so the JavaPlex tutorial's own "pick landmarks, read R , use $2R$ as the threshold" recipe isn't reproducible through it. `matlab.TDA4j` also offers the SAME construction split into two steps for exactly this: `selectLandmarksFrom{Points,DistanceMatrix}` (step 1, returns a `LandmarkSelectionResult` with both `landmarks()` and `coveringRadius()`) and `computeFrom{Points,DistanceMatrix}AndLandmarks` (step 2, takes that landmark array directly). Both entry points work from Scala too, not just MATLAB:

```
val selection
= TDA4j.selectLandmarksFromPoints(points, Array("numLandmarks", "50"))
val result = TDA4j.computeFromPointsAndLandmarks(
  points,
  selection.landmarks(),
  Array("maxFiltrationValue", (2 * selection.coveringRadius()).toString)
)
```

From MATLAB:

```
javaaddpath('target/scala-3.9.0/TDA4j-<version>-assembly.jar');
selection =
  org.appliedtopology.tda4j.matlab.TDA4j.selectLandmarksFromPoints(points,
    {'numLandmarks', '50'});
landmarks = selection.landmarks(); % int32 array, 0-BASED ambient indices
into `points`
R = selection.coveringRadius();

result =
  org.appliedtopology.tda4j.matlab.TDA4j.computeFromPointsAndLandmarks(points,
    landmarks, ...
    {'maxFiltrationValue', sprintf('%.17g', 2 * R)});
bars = result.toArray();
```

Two MATLAB-specific traps worth calling out explicitly:

- **landmarks is 0-based** (matching `cycleVertices`'s own convention), but MATLAB arrays are 1-based — index points with `points(landmarks + 1, :)`, not `points(landmarks, :)`.
- Use `sprintf('%.17g', 2 * R)`, not `num2str(2 * R)`, to build the `maxFiltrationValue` option string. `num2str`'s own default precision silently rounds most covering radii, which can shift which simplices actually fall under the threshold — `sprintf('%.17g', ...)` round-trips a double exactly.

`coveringRadiusFromPoints/coveringRadiusFromDistanceMatrix` compute `R` for a landmark set you didn't get from `selectLandmarksFrom*` (hand-picked, or reused from elsewhere) — same `2R` recipe, different source for the landmarks. Every entry point in this two-step family validates its own `landmarks` array eagerly (non-empty, 0-based in range, no duplicates), with a message that specifically flags an index equal to `points.length` as a likely 1-based-indexing mistake.

From the CLI, the same two steps are two separate invocations:

```
tda4j --select-landmarks --num-landmarks 50 --output landmarks.txt points.csv
# tda4j: covering radius R = 0.12081466774937936 -- e.g. pass
'0.24162933549875873' as --max-filtration-value...
```

```
tda4j --landmarks-file landmarks.txt --complex witness --max-filtration-value
0.24162933549875873 points.csv
```

The first writes one 0-based landmark index per line to `landmarks.txt` (plus a `# coveringRadius=...` comment line) and prints `R` — and the exact `2R` string to pass on — to `stderr`; **use that printed string verbatim**, not a value you round or retype by hand (the same truncation trap `sprintf("%.17g")` avoids above applies here too). The second invocation reads the landmarks file back in and computes the barcode. `--complex witness` on the second line is optional (that combination of flags can only ever mean a witness complex) but accepted if you type it out of habit from the one-shot form.

Cubical complexes and images

```
val stream = CubicalImage.fromFlatArray(shape = IndexedSeq(3, 3), flatValues
= pixelValues, sublevel = true)
val homology = CubicalHomologyContext[Double, Double]
().persistentHomology(stream)
homology.diagramAt(Double.PositiveInfinity)
```

`CubicalImage` also has `fromGrayscale2D`, `fromVoxelGrid3D`, `fromBufferedImage`, and `fromFile` for loading real images/volumes. `sublevel = false` computes superlevel-set persistence instead (ascending vs. descending intensity) via the standard "negate the values" trick — reported filtration values under `sublevel = false` are in negated-intensity units, not raw pixel values.

Simplicial sets

For homology of a space presented combinatorially (not as a metric-space complex), build a `FiniteSimplicialSet[G]` by giving each generator's faces directly. A minimal circle (one vertex, one loop edge):

```
enum CircleGen { case V, E }
import CircleGen.*
given Ordering[CircleGen] = Ordering.by(_.ordinal)

val circle = FiniteSimplicialSet[CircleGen](summon[Ordering[CircleGen]])(
  generatorsByDim = IndexedSeq(Set(V), Set(E)),
  faces = {
    case V => IndexedSeq.empty
    case E => IndexedSeq(SSetElement(Nil, V), SSetElement(Nil, V)) // both
    faces of the loop are V
  }
)
circle.validate() // Seq.empty -- no errors

given (CircleGen is OrderedCell) = circle.cellInstance
```

```
val stream = FilteredSimplicialSetStream(circle, { case V => 0.0; case E => 1.0
})
CellularHomologyContext[CircleGen, Double, Double]()
  .persistentHomology(stream)
  .diagramAt(Double.PositiveInfinity)
// List((1, 1.0, Infinity), (0, 0.0, Infinity)) -- H0 = H1 = one essential
// class each, as expected for S^1
```

`FiniteSimplicialSet`'s companion object builds new simplicial sets from existing ones instead of by hand: `FiniteSimplicialSet.product/.coproduct` (the categorical product/coproduct) and `.quotient/.identify` (attaching maps — glue generators together, or collapse one down onto a lower-dimensional target). `validate()` checks that hand-written or constructed face data actually satisfies the simplicial identities; it's a necessary sanity check, not proof the resulting space is the one you intended.

Loading and saving data: the io module

`org.appliedtopology.tda4j.io` reads and writes the file formats the wider TDA ecosystem uses, so you don't have to hand-roll parsing:

Object	Formats
CSV	plain CSV point clouds, full/lower-triangular distance matrices, persistence diagrams
Ripser	Ripser's point-cloud/lower/upper/full/binary distance-matrix formats
Dipha	DIPHA's distance-matrix, cubical-image, and persistence-diagram formats
Gudhi	GUDHI's OFF point-cloud format and persistence-diagram format
Perseus	Perseus's cubical topox and persistence-interval formats

Each object offers a raw loader (`readPointCloud/readFullDistanceMatrix/...`) and a one-line convenience constructor on top (`readEuclideanMetricSpace/readExplicitMetricSpace/readCubicalGridStream/...`):

```
import org.appliedtopology.tda4j.io.{given, *}

val metricSpace = Ripser.readEuclideanMetricSpace("points.txt")
val stream = Perseus.readCubicalToplex("image.txt")
```

Two format details worth knowing if you're comparing output against another tool: Ripser's binary distance-matrix format is 32-bit `float`, not 64-bit `double`. DIPHA's and Perseus's cubical-grid axis order is the opposite of CubicalImage's own convention (first axis fastest-varying vs. last); both readers/ writers here handle the transposition for you.

Command-line tool: `tda4j`

`sbt assembly` builds a runnable fat jar exposing the whole library as a command-line tool, without writing any Scala:

```
java -jar target/scala-3.9.0/TDA4j-<version>-assembly.jar [options] <input-file>
```

It loads a point cloud, distance matrix, or cubical image in one of several formats (`--input-format`), computes persistence via the same facade the MATLAB bridge uses (below), and writes the result in one of several formats (`--output-format`: `text`, `csv`, `gudhi`, `dipha`, `perseus`). Run with `--help` for the full flag list; the main ones mirror the MATLAB options one-to-one: `--complex` (`vr/alpha/cech/ witness/dtm-rips/dtm-alpha/sheehy-rips`), `--engine`, `--max-dimension`, `--max-filtration-value`, `--field`, `--representatives` (also print each bar's representative chain), and (for `--complex=witness`) `--num-landmarks`, `--witness-variant`, `--landmark-selector`, `--landmark-seed`, `--nu`. For `--complex=dtm-rips` or `--dtm-alpha`, use `--dtm-k` (required), `--dtm-q` (default 2.0), and `--dtm-p` (default 1.0, only for `dtm-rips`). For `--complex=sheehy-rips`, use `--sheehy-epsilon` (required, strictly between 0 and 1). `--select-landmarks/--landmarks-file` split that same witness-complex computation into the two-step recipe described above.

Calling from MATLAB or Java

`org.appliedtopology.tda4j.matlab.TDA4j/PersistenceResult` is a real, tested facade for calling TDA4j from MATLAB's built-in Java interface, or from any plain-Java caller — every public method and return type is a plain `int`, `double`, `String`, `double[][]`, or `String[]`; no Scala types, no generics, no context parameters, no Unicode operator names.

```
double[][] points = { {0.0, 0.0}, {1.0, 0.0}, {0.5, 0.8} };
PersistenceResult result = TDA4j.computeFromPoints(points);

double[][] bars = result.toArray(); // one row per bar: [dimension, birth,
    death], death = Inf if essential
double[] coeffs = result.cycleCoefficients(0);
int[][] simplices = result.cycleVertices(0); // each row: a simplex's sorted
    vertex indices
```

```
javaaddpath('target/scala-3.9.0/TDA4j-<version>-assembly.jar');
points = [0.0 0.0; 1.0 0.0; 0.5 0.8];
result = org.appliedtopology.tda4j.matlab.TDA4j.computeFromPoints(points);
bars = result.toArray();
```

Entry points: `computeFromPoints/computeFromDistanceMatrix` (Vietoris-Rips/alpha/Cech/witness/dtm-rips/ dtm-alpha/sheehy-rips, from a point cloud or a precomputed distance matrix — alpha, Cech, and dtm-alpha need real coordinates, so they're only available from the points overload; witness/dtm-rips/sheehy-rips work from either, exactly like `vr`, since none of the three needs real coordinates, only a metric), `computeFromCubicalImage/computeFromImage` (cubical persistence from a flat array + shape, or a 2D pixel matrix directly), and the two-step witness recipe's own four entry points -- `selectLandmarksFromPoints/selectLandmarksFromDistanceMatrix` (# `LandmarkSelectionResult`) and `computeFromPointsAndLandmarks/computeFromDistanceMatrixAndLandmarks`, plus the `coveringRadiusFromPoints/coveringRadiusFromDistanceMatrix` query pair -- covered in their own section above rather than the table below, since they take an explicit `int[] landmarks` parameter and each has its OWN (stricter) recognized-options set rather than sharing the table's. Every method here has a no-options overload and one taking a flat, alternating key/value `String[]` of options — so adding a new option in the future never changes a method's call signature:

Option	Values	Default
<code>complex</code>	<code>vr</code> , <code>alpha</code> , <code>cech</code> , <code>witness</code> , <code>dtm-rips</code> , <code>dtm-alpha</code> , <code>sheehy-rips</code>	<code>vr</code>
<code>engine</code>	<code>ripser</code> , <code>naive</code> , <code>chunks</code> , <code>cohomology</code>	<code>ripser</code> for <code>vr</code> and <code>witness/witnessVariant=lazy</code> ; <code>naive</code> for <code>alpha/cech/dtm-rips/dtm-alpha/sheehy-rips/witness/witnessVariant=general</code>
<code>alphaBackend</code>	<code>helix</code> , <code>DQP</code>	<code>helix</code> (only consulted for <code>complex=alpha</code>)
<code>dtmK</code>	integer	REQUIRED for <code>complex=dtm-rips</code> or <code>complex=dtm-alpha</code> , no default

Option	Values	Default
dtmQ	double	2.0 (only consulted for complex=dtm-rips or complex=dtm-alpha)
dtmP	double	1.0 (only consulted for complex=dtm-rips; must be 1.0 or 2.0)
sheehyEpsilon	double	REQUIRED for complex=sheehy-rips, no default; strictly between 0 and 1
maxDimension	integer	2 — highest H _k reported, not highest simplex dimension built
maxFiltrationValue	double	the point cloud's own minimum enclosing radius (+Infinity for witness/witnessVariant=general; SheehyRipsSimplexStream's own maxFiniteFiltrationValue for complex=sheehy-rips)
field	Z (finite field), R (floating point)	Z, prime=2
prime	integer	2 (only for field=Z)
epsilon	double	1e-9 (only for field=R; unrelated to sheehyEpsilon above)
numLandmarks	integer	REQUIRED for complex=witness, no default
witnessVariant	lazy, general	lazy (only consulted for complex=witness)

Option	Values	Default
<code>landmarkSelector</code>	<code>maxmin, random</code>	<code>maxmin</code> (only consulted for <code>complex=witness</code>)
<code>landmarkSeed</code>	<code>integer</code>	<code>0</code> (only for <code>complex=witness/landmarkSelector=random</code>)
<code>nu</code>	<code>0, 1, 2</code>	<code>2</code> (only for <code>complex=witness/witnessVariant=lazy</code>)

`alpha` refuses `engine=ripser` and `engine=chunks` (neither engine understands `alpha` complexes, and the `chunks/alpha` combination is a known stall risk in the underlying library); `cech`, `dtm-rips`, and `sheehy-rips` all refuse `engine=ripser` (the packed Ripser engine's optimizations are proven for Vietoris-Rips's plain max-pairwise-distance functional specifically, not for Cech's circumradius, DTM's weighted filtration, or Sheehy's sparsified/vanishing one); `dtm-alpha` refuses both `engine=ripser` and `engine=chunks`; `witness` with `witnessVariant=general` refuses both `engine=ripser` and `engine=chunks` for the same reason as `cech` (the general witness complex isn't a flag complex either) — use `witnessVariant=lazy` (the default) for `engine=ripser/chunks`. `engine=cohomology` is accepted everywhere `engine=naive` is (`vr`, `alpha`, `cech`, `dtm-rips`, `dtm-alpha`, `sheehy-rips`, and `witness` alike). See the [Developer's Guide's](#) `streams-vs-engines` table for the full picture, complex by complex. Unrecognized keys or values throw `IllegalArgumentException` immediately rather than silently falling back to a default.

Representative-chain vertex indices for `complex=witness` are **ambient point-cloud indices**, already mapped back from the stream's own local `0` until `numLandmarks` landmark indices — `cycleVertices` never reports a raw local landmark index.

`PersistenceResult.cycleVertices/cycleCoefficients` give you each bar's representative chain: every engine records one for every bar — though for `engine=cohomology`, only an *essential* bar's representative is guaranteed to be a genuine cocycle (zero coboundary); a finite bar's is a valid witness on its own living interval, not over the whole complex (see the developer's guide's `persistence-engines` page for why).

Which persistence engine?

Need	Engine (engine= for MATLAB/CLI)
Exploration, intermediate-filtration queries, representative cycles	<code>naive</code> (<code>CellularHomologyContext/TDAContext</code>)
Fastest, most memory-efficient — the default for <code>complex=vr</code>	<code>ripser</code> (<code>PackedRipserCohomologyContext</code>)
Large complex, want representatives for every bar including essential ones	<code>chunks</code> (<code>CellularPersistenceInChunksContext</code>)
Cohomology (cocycle representatives) on <code>Cube/FiniteSimplicialSet</code> , or on <code>Alpha/Cech/DTM/Sheehy/witness</code> , where <code>ripser</code> doesn't apply	<code>cohomology</code> (<code>CellularCohomologyContext</code>)
<code>Alpha</code> or <code>Cech</code> or <code>DTM</code> or <code>Sheehy</code> complexes, or a general (non-flag) witness complex	<code>naive</code> or <code>cohomology</code> (<code>chunks</code> also works for <code>Cech</code> , <code>DTM-Rips</code> , and <code>Sheehy-Rips</code> — not <code>Alpha/DTM-Alpha</code>)
A lazy witness complex (the flag-complex variant)	<code>ripser</code> (<code>PackedRipserCohomologyContext</code> , run directly on <code>WitnessMetricSpace</code>) or <code>naive/chunks/cohomology</code>

All engines are generic over the coefficient field (a prime finite field or floating point); `naive`, `chunks`, and `cohomology` are also generic over the cell type (simplices, cubes, or simplicial-set generators) — only `ripser` is Vietoris-Rips-specialized. See the [Developer's Guide's persistence-engines page](#) for the full detail.

Which alpha-complex backend?

- `"helix"` (`HelixDelaunay`) — an actual Delaunay triangulation, computed incrementally. What `dispatch = "default"` resolves to. Has a known, quantified failure mode on point clouds with a near-cospherical local cluster: zero failures across 20,000-trial fuzz testing at ambient dimension 2 and 5, but roughly 1-in-170 at ambient dimension 4 with 20-30 points, on ordinary-looking input. Don't treat its output as unconditionally reliable ground truth at ambient dimension 4 or higher.
- `"DQP"` (`AlphaShapeDQP`) — a dual active-set quadratic-programming method (Carlsson & Carlsson 2024) that never builds a Delaunay triangulation at all. Its real strength is high ambient dimension,

where Delaunay becomes infeasible, and exact homology rather than an approximate persistence diagram. The paper's own published benchmarks are mixed — it loses to Ripser on 2 of 4 of its own persistence examples, and to qhull-based Delaunay on some inputs. The honest value proposition is high-dimensional feasibility and exactness, not raw speed.

Both backends agree that in degenerate (cospherical) point configurations — e.g. points on a regular grid — the alpha complex genuinely contains higher-dimensional simplices than a triangulation-based mental model would suggest (a unit grid in the plane produces 3-simplices, one per unit square, not just triangles). This is correct behavior, not a bug in either backend; users coming from CGAL or GUDHI may find it surprising.

Performance: opt-in parallelism

A few of the more expensive per-cell computations can run on the JVM's common thread pool, opt-in via a constructor flag (`AlphaDQPSettings.parallel`, `CubicalGridStream.parallelFiltrationValue`, `CechCofaceSimplexStream.parallelFiltrationValue`), all defaulting to `false`, with deterministic output either way. Worth turning on for a large alpha-complex computation (each vertex's own QP solve is genuinely expensive — measured 2-4x speedup at a few hundred points and above); cubical images and Cech complexes see a smaller win (a few percent) since the per-cell cost there is lighter.

Tutorials

Tutorials — currently a placeholder; porting Henry Adams' JavaPlex tutorials to TDA4j is tracked there as future work, not yet done. The witness-complex construction those tutorials lean on heavily is now implemented (`LandmarkSelector/WitnessGeometry/LazyWitnessSimplexStream/WitnessCofaceSimplexStream`, see "Witness complexes" above) — a building block for that port, not the port itself.

Developer's Guide for TDA4j

TDA4j is meant to be a viable platform for algorithm development and research into topological data analysis, not just a library you call into as a black box. This guide gets you from "I understand persistent homology" to "I can confidently write, review, and extend code in this repository" — which, if Scala 3.7+'s newest context-abstraction syntax is new to you (as it is for most people — it's genuinely recent), is most of the actual gap.

How to use this guide

1. **The Scala 3 syntax.** `Type` is `TypeClass`, `type Self: Ordering` as `ordering`, `given [bounds] => Body = value`, opaque types, extension methods. None of this is older-Scala-3-style code that "should" already look familiar. Start with the [Scala 3 primer](#).
2. **Where a given piece of functionality lives, and what layer it's in.** The library is built in clear layers — algebra, then complex construction (streams), then the persistence engines that consume a stream — and each has its own subpackage. See [Architecture](#).
3. **Which persistence algorithm, and which of its implementations, to build on.** `homology` ships four independently-implemented algorithms across five concrete classes; they are not layers on one shared core, and their intended roles (production vs. reference oracle) are not something you can infer just because all five compile and share a file. See [Persistence engines](#).
4. **What traps look fine, compile fine, and only produce a wrong answer on specific inputs.** Ordering and reduction code in this codebase has a real history of exactly this failure mode. See [Hard-won invariants](#) before you trust your own "this looks obviously correct" judgment on a change to reduction or ordering logic.
5. **Whether a surprising result is a bug or the mathematically correct answer on a degenerate input.** See [Degeneracies](#) before "fixing" something that isn't broken.

Naming convention: `tda4j`/`TDA4j`, never `Tda4j`

"TDA4j" is the acronym TDA ("topological data analysis") plus the conventional `4j` suffix (`log4j`, `slf4j`) — not an ordinary English word, so don't let a titlecasing habit turn it into `Tda4j`. Use `tda4j` (all lowercase) for the package name, the executable/artifact name, the repo name, and ordinary prose; use `TDA4j` (acronym preserved) for Scala identifiers that must start with a capital letter — class/object/trait names such as `TDA4j` (the MATLAB/Java facade), `TDA4jConf`/`TDA4jCLI` (the CLI). `Tda4j` is never correct.

The same rule applies to any identifier that's itself an acronym: `io.CSV` (comma-separated values) and `cli.TDA4jCLI` (command-line interface) are both fully capitalized for the same reason. `io.Gudhi`/`Dipha`/

Ripser/Perseus are ordinary proper nouns (the external projects those file formats belong to) and stay titlecased in their own conventional spelling instead.

A Scala 3.7+ primer for this codebase

If you know topology cold but Scala 3's newest context-abstraction features are new to you, read this before anything else. TDA4j leans hard on Scala 3.7+ syntax that didn't exist a few years ago, and code samples you find by searching the web (or asking an LLM trained on older Scala) will often look superficially similar but be subtly wrong for this codebase. Every construct below is demonstrated with real code from the current source tree, not a simplified stand-in.

The problem all of this solves

TDA4j wants to write code like "any `CellT` that has a `boundary` operation and a total order can be fed to the homology machinery" — generic over the *shape* of a capability, not over a fixed class hierarchy. Scala's traditional answer to this is the typeclass pattern: a trait describing the capability, and `given/implicit` instances witnessing that a particular type has it. Scala 3.5+ added a second, newer way to write typeclasses (sometimes called "type member style" or "modularity" style, following SIP-64, <https://docs.scala-lang.org/sips/sips/typeclasses-syntax.html>, and described further at <https://dotty.epfl.ch/docs/reference/experimental/typeclasses.html>). TDA4j uses this newer style for its *own* typeclasses (`RingModule`, `Field`, `Cell`, `OrderedCell`, ...) while still using the traditional `Ordering[T]`-style parametrized typeclasses for standard-library ones. You need to be fluent in reading both.

Context bounds, old and new

A context bound like `[CellT: Ordering]` has existed in Scala for a long time — it's sugar for an implicit/given parameter (using `Ordering[CellT]`). Two things changed recently that show up constantly in this codebase:

Named context bounds (as). Instead of writing a context bound and then separately summoning it, you can name the bound's evidence inline:

```
trait OrderedCell extends Cell:
  type Self: Ordering as ordering
```

(`Chain.scala`.) This says "Self has an `Ordering`, and I'll call that ordering instance `ordering` inside this trait body" — no `summon[Ordering[Self]]` needed anywhere below. The same pattern appears on plain type parameters too, e.g. `Chain.apply[CellT: Ordering as ord, ...]` in `Chain.scala`'s `from`.

Multiple bounds in one clause. `type Self: {Ordering, Filterable}` (seen in `SimplexStream.scala`'s `Filtration` trait) bounds one type by several typeclasses at once, replacing the older, clunkier `[T: Ordering: Filterable]` chained-colon style.

Type is TypeClass: the typeclass-as-type-member pattern

This is the single most important piece of syntax to internalize before reading `Chain.scala` or `Homology.scala`. A typeclass in this style is a trait with an abstract *type member* called `Self`, rather than a type parameter:

```
trait HasDimension:
  type Self
  extension (self: Self) def dim: Int

trait Cell extends HasDimension:
  type Self
  extension
    (self: Self) def boundary[CoefficientT: Field]: Seq[(Self, CoefficientT)]

trait OrderedCell extends Cell:
  type Self: Ordering as ordering
```

(`algebra/Chain.scala`, current source — note `boundary` returns `Seq[(Self, CoefficientT)]`, not a `Chain`; more on that in [Architecture](#).)

To say "type `T` implements `Cell`," you don't write `Cell[T]` — you write `T is Cell`, using the special infix type alias `is` that the typeclasses feature provides (roughly `infix type is[A, C <: {type Self}] = C {type Self = A}` — a refinement type under the hood, not a real generic parameter). You'll see this both as a type (`Chain[CellT, CoefficientT] is RingModule`, a type used all over `Homology.scala`) and as a context bound (`[CellT: OrderedCell]`, which desugars to "there exists a given instance of `OrderedCell` { `type Self = CellT` }" — the compiler treats both spellings, `OrderedCell[CellT]`-shaped and `CellT is OrderedCell`-shaped, as satisfying the same bound).

A concrete instance looks like this — `SimplexOrderedCell.scala`'s actual construction of `Simplex[VertexT] is OrderedCell`:

```
given defaultSimplexIsOrderedCell: [VertexT: Ordering] =>
  (Simplex[VertexT] is OrderedCell) =
    simplexIsOrderedCell[VertexT]()
```

where `simplexIsOrderedCell` builds an anonymous new `(Simplex[VertexT] is OrderedCell)`: instance, providing the `ordering` member `OrderedCell` requires and, in an extension (`spx: Simplex[VertexT]`) block, concrete implementations of `dim` and `boundary`. Once this given is in scope, any `Simplex[Int]` value can call `.dim` and `.boundary[Double]` directly, with no explicit typeclass-dictionary plumbing at the call site — that's the entire payoff of the pattern. **One gotcha specific to this codebase's own package `split`:** a given only comes into scope via a wildcard import that

explicitly says so — `import org.appliedtopology.tda4j.cells.{given, *}`, not just `import org.appliedtopology.tda4j.cells.*`. A plain `import pkg.*` does **not** bring `given` instances into scope in Scala 3; every file in this codebase that reaches across a subpackage boundary uses the `{given, *}` form for exactly this reason — see [Architecture](#)'s package-layout section.

One syntax detail worth flagging because it trips people up: `given [CellT: OrderedCell as oCell] => Ordering[CellT] = oCell.ordering(algebra/Chain.scala)` is the "anonymous given via arrow" form — a `given` with no name, whose *value* is given after `=>`, parametrized by a context-bound clause on the left of the arrow. Read `given [bounds] => Body = value` as "for any type satisfying `[bounds]`, here is a value of type `Body`." This is how the library bridges its own `OrderedCell.ordering` member back into the standard-library `Ordering[T]` typeclass, so anything that's `OrderedCell` automatically also participates in ordinary Scala code (`.sorted`, `SortedMap`, etc.) that expects a plain `Ordering`.

Extension methods

`extension (self: Self) def dim: Int` inside a trait declares an *abstract* extension method; a concrete `given` instance provides the method body in its own `extension (...)` block. Outside of typeclass bodies, plain top-level extension blocks are used too, e.g. `SimplexOps.scala`'s entire file is one big extension `[VertexT](spx: Simplex[VertexT])` block delegating most of `SortedSet`'s API (`.size`, `.map`, `.filter`, `.union`, ...) to `spx.underlying`. This is why `Simplex` "feels like" a `SortedSet` even though, as the next section explains, it technically isn't one at runtime.

Opaque types

```
opaque type Simplex[VertexT] = SortedSet[VertexT]
```

(`Simplex.scala`.) `Simplex[VertexT]` is *defined to be* `SortedSet[VertexT]` at the bytecode level — zero runtime wrapper, zero boxing overhead — but outside `Simplex.scala` itself, the compiler treats `Simplex[VertexT]` and `SortedSet[VertexT]` as distinct, incompatible types. This buys type safety (you can't accidentally pass a raw `SortedSet[Int]` where a `Simplex[Int]` is expected, so a caller can't build a `Simplex` bypassing whatever invariants its constructors enforce) at zero runtime cost. The two extension methods `.underlying` and `.asSimplex` (`Simplex.scala:19,21`) are the sanctioned crossing points between the two views — every other file that needs to fall back to raw `SortedSet` operations goes through `.underlying`, never an unsafe cast.

Practical consequence for writing new code: you cannot pattern-match structurally on a `Simplex`'s contents without going through its extension-method API or the provided `unapplySeq`

(`Simplex.from(...)` constructs; `Simplex.unapplySeq` deconstructs, letting you write `case Simplex(a, b, c) => ...`).

given/summon resolution: static, not dynamic — and why this matters more than usual here

This is ordinary Scala 3 implicit-resolution behavior, but it is unusually load-bearing in this codebase, enough that it has caused real, confirmed bugs (see [Hard-won invariants](#) for the full story). The short version: when a `given` instance's body summons another `given` (e.g. `Chain[CellT, CoefficientT]` is `RingModule`'s implementation needs an `Ordering[CellT]` in scope to build its internal `SortedMap/PriorityQueue`), that inner summon happens **once, when the outer given is first constructed** — not fresh on every later call to the outer instance's methods. If you construct a `given` too early, before the specific `Ordering` you actually wanted is in scope, the instance permanently closes over whatever fallback was in scope at that moment, and no later import or context change will fix it. This is exactly why every `HomologyState` in `homology/Homology.scala` declares `given Ordering[CellT] = stream.filtrationOrdering` as its *first* line, before summoning `Chain[CellT, CoefficientT]` is `RingModule` on the next line — order matters, and it isn't stylistic.

What to read next

[Architecture](#) walks the algebraic core and complex-construction layers using the concepts above; [Persistence engines](#) covers the four homology algorithms and their trust status; [Hard-won invariants](#) is the concentrated list of non-obvious rules (like the `given`-timing one above) that this codebase depends on and that are easy to violate even once you know they exist.

Architecture: from algebra to a filtration stream

This page walks the two lower layers of the library: the algebraic core (what a "chain" and a "coefficient" actually are) and complex construction (how a sequence of cells in filtration order gets produced). **Persistence engines** covers what consumes the stream this layer produces.

If a piece of Scala 3 syntax below looks unfamiliar, see the **Scala 3 primer** first.

Package layout

`org.appliedtopology.tda4j` is split into subpackages, each a layer:

- `algebra` — `RingModule`, `Field`, `FiniteField`, `Chain` (including the `Cell/OrderedCell/OrderedBasis` contracts), `SSetElement` (the degeneracy-word algebra underlying simplicial sets). The typeclasses and formal-sum machinery everything else builds on.
- `cells` — `Simplex`, `Cube`, `FiniteSimplicialSet` — the three concrete `OrderedCell` instances — plus `FiniteSimplicialSet`'s companion (`product/coproduct/quotient/identify`), which draws on shared ordering helpers kept in the separate `SimplicialSetConstructions.scala`.
- `streams` — everything that produces cells in filtration order: `SimplexStream/CellStream`, the Vietoris-Rips family, `FiniteMetricSpace`, `CubicalStream/CubicalImage`, `SimplicialSetStream/FilteredSimplicialSetStream`, `CechStream`, `WitnessStream`, `UnionFind`.
- `homology` — the persistence algorithms (`Homology.scala`, `PackedRipserCohomology.scala`, `Cohomology.scala`).
- `barcode` — `Barcode`, `PersistenceBar`, `BarcodeEndpoint`.
- `alpha` — `AlphaShapes` (`HelixDelaunay/AlphaShapeDQP`), `AlphaComplexDQP`.
- `io` — file-format adaptors: `CSV`, `Ripser`, `Dipha`, `Gudhi`, `Perseus`.
- `cli` — the `tda4j` executable (`TDA4jConf`, `TDA4jCLI`), a thin translator over `matlab.TDA4j/io`.
- `matlab` — `TDA4j/PersistenceResult/LandmarkSelectionResult`, the plain-primitives facade for MATLAB and other Java callers.
- `root` (`org.appliedtopology.tda4j` itself) — `package.scala` (`TDAContext`), the user-facing Scala facade.

Load-bearing import rule: every file that reaches across a subpackage boundary does it via `import org.appliedtopology.tda4j.<pkg>.{given, *}` — the `given` matters. A plain `import pkg.*` does **not** bring `given` instances into scope in Scala 3, and this codebase's `Ordering/RingModule/Field` instances are all `givens`. Forgetting `given` compiles cleanly and fails at a summon site with a confusing "no given instance" error far from the missing import.

The algebraic core

RingModule and Field

`algebra/RingModule.scala` defines what it means for a type `Self` to be a module over a ring-like type `R`:

```
trait RingModule:
  type Self
  type R
  def zero: Self
  def plus(x: Self, y: Self): Self
  def minus(x: Self, y: Self): Self = plus(x, negate(y))
  def negate(x: Self): Self = minus(zero, x)
  def scale(x: R, y: Self): Self
  extension (t: Self)
    def +(rhs: Self): Self = plus(t, rhs)
    def -(rhs: Self): Self = minus(t, rhs)
    def <*(rhs: R): Self = scale(rhs, t)
    def unary_- : Self = negate(t)
  extension (r: R)
    def |(t: Self): Self = this.scale(r, t)
    def #(t: Self): Self = this.scale(r, t) // "boxed times" -- scalar action
```

A minimal instance only needs `zero`, `plus`, `scale`, and one of `minus`/`negate`. `#` (U+22A0, typed via a Unicode input method or copy-paste) is the scalar-multiplication operator used throughout: `2.0 # chain`.

`algebra/Field.scala` is a separate, self-contained typeclass (not built on `RingModule`) for coefficient types themselves — `plus/minus/times/divide/negate/invert/zero/one`, with `+/-/*/eq1` extension operators. **There is no default given** `Double is Field anywhere in src/main` — bring one in explicitly:

```
given Double is Field = Field.DoubleApproximated(1e-9)
```

`Field.DoubleApproximated(epsilon)` treats two doubles as equal within `epsilon` — necessary because exact floating-point equality is rarely what you want when an `isZero` check decides whether a chain entry survives a reduction step. `FiniteField.scala` gives exact, non-approximate coefficients

instead: `FiniteField(p)`'s `Fp` is an opaque type *per instance* (so `Fp` from a mod-5 field and a mod-7 field are distinct, incompatible types), with exact arithmetic via a precomputed inverse table.

Cell, OrderedCell, and what boundary returns

```
trait Cell extends HasDimension:
  type Self
  extension
    (self: Self) def boundary[CoefficientT: Field]: Seq[(Self, CoefficientT)]

trait OrderedCell extends Cell:
  type Self: Ordering as ordering
```

`boundary` returns a plain `Seq[(Self, CoefficientT)]`, not a `Chain`. Callers wrap the result in `Chain.from(...)` when they need actual `Chain` machinery.

`Simplex[VertexT]`, `Cube`, and `FiniteSimplicialSet[G]`'s generators are the library's three concrete `OrderedCell` instances (`cells/SimplexOrderedCell.scala`, `cells/CubicalOrderedCell.scala`, `cells/SimplicialSet.scala`): `boundary` returns each codimension-1 face paired with alternating signs.

There is no dual `Cocell/OrderedCocell` trait pair — an earlier version of this codebase had one, and it was removed: `coboundary` is *extrinsic* to a cell (it depends on which higher-dimensional cells actually exist in the ambient, possibly-truncated complex), not intrinsic the way `boundary` is, so a per-cell `coboundary` method with no complex to consult can only be correct when the complex is always the full combinatorially-possible one. `RipserCohomologyContext/PackedRipserCohomologyContext` compute coboundaries directly against `SimplexIndexing`'s cofacet iterator instead (see [Persistence engines](#)).

Chain

`Chain[CellT: Ordering, CoefficientT: Field]` (`algebra/Chain.scala`) is a formal sum of cells with field coefficients, backed by a mutable `PriorityQueue` ordered so the *smallest* cell under the ambient `Ordering[CellT]` sits at the head — cheap to peek, since "leading term" (the reduction pivot) is queried constantly.

- `collapseHead()`/`collapseAll()` merge duplicate-cell entries, dropping exact zeros. Naive `+/-/#` only lazily collapse the head, so a hand-rolled reduction loop built out of raw `Chain` arithmetic accumulates an ever-growing backlog of uncollapsed duplicates — see [Hard-won invariants](#).
- `Chain.reduceBy/Chain.reduceByUntil` are the actual matrix-reduction primitives every persistence engine builds on: given a chain `z` and a basis: `Map[CellT, Chain[CellT, CoefficientT]]` of recorded pivot columns, repeatedly subtract the appropriate multiple of

`basis(pivot)` until `z`'s leading cell is no longer a key in `basis`, returning both the reduced chain and a reduction-log chain (the multipliers used) that lets a caller reconstruct a V-column. Internally these go through a `mutable.TreeMap`, not the `PriorityQueue`-backed `Chain` type, so repeated updates collapse duplicates automatically.

- given `[CellT: Ordering, CoefficientT: Field] => (Chain[CellT, CoefficientT] is RingModule)` is what makes `+`, `-`, `#`, `unary_` - work on `Chain` values — the given whose summon *timing* matters, see **Hard-won invariants**.

Complex construction: streams

A `CellStream[CellT, FiltrationT]` is the abstract interface every persistence engine consumes: an iterator over cells in filtration order, plus a `filtrationValue: PartialFunction[CellT, FiltrationT]` and a `Filterable` (smallest/largest sentinel values, `±Infinity` for `Double`). Trait hierarchy, most-general to least:

```
Filtration[CellT, FiltrationT]      -- has a filtrationValue partial
  function
CellStream[CellT, FiltrationT]      -- Filtration + IterableOnce[CellT] +
  filtrationOrdering
SimplexStream[VertexT, FiltrationT] -- CellStream specialized to
  Simplex[VertexT]
StratifiedCellStream[CellT, FiltrationT] -- adds iterateDimension: dimension-
  by-dimension access
```

Two rules that hold for every stream in this codebase, both explained in full in **Hard-won invariants**:

- `filtrationOrdering` orders cells by filtration value **reversed** (so the *smallest*-under-this-ordering cell is the *youngest*), because `Chain`'s pivot is always the ordering's minimum and the reduction algorithms need "leading cell" to mean "youngest." A stream's iteration order and its `filtrationOrdering` must be the *same* total order (one the consistent `.reverse` of the other), not merely two independently valid orderings.
- `StratifiedCellStream.iterateDimension`'s domain must be contiguous from `0` (defined for `0, ..., k` or all of `#`, never with a gap) — the default `.iterator` stops at the first dimension it's undefined for.

Vietoris-Rips: several independent generation strategies

These are **alternate engines with the same output contract, not layers on one another** — check which `iterateDimension/iterator` implementation is actually in play before reasoning about a bug:

- `EnumeratingCofaceSimplexStream` — generates each dimension explicitly via `SimplexIndexing`'s combinatorial-number-system enumeration, filtered by an optional `keepCriterion`.
- `RipserCofaceSimplexStream` — a coface-generation variant that only expands cofaces reachable from the previous dimension's survivors rather than re-enumerating a whole dimension from scratch. `CechStream`'s `CechCofaceSimplexStream` is built directly on top of this class (see below).
- `InorderCofaceSimplexStream` — generates cofaces "in order" directly from the metric-space structure.
- `IncrementalVietorisRipsSimplexStream` — a reference implementation of Antonio Rieser's New-VR algorithm (arXiv:2301.07191), meant as a cross-validation baseline rather than a speed-competitive engine.
- `RecursiveStackVietorisRipsSimplexStream` — a third, independent coface-enumeration strategy built on a recursive stack and spatial query.

`EnumeratingCofaceSimplexStream/RipserCofaceSimplexStream/InorderCofaceSimplexStream/IncrementalVietorisRipsSimplexStream` all default `maxFiltrationValue` to `metricSpace.minimumEnclosingRadius`, not `Double.PositiveInfinity` — past that radius every vertex is within range of a common apex, so the complex is a cone from there on and provably contributes no further homology (real `ripser.cpp` uses the same default). Pass `maxFiltrationValue = Some(Double.PositiveInfinity)` for the old always-unbounded behavior. `RecursiveStackVietorisRipsSimplexStream` and the alpha-complex backends keep their own, always-untruncated default (see [Alpha complex](#)).

`SimplexIndexing` (inside `SimplexIndexing.scala`) is the piece every Ripser-flavored part of this codebase depends on: it encodes/decodes a d -subset of $\{0, \dots, \text{vertexCount}-1\}$ to/from a single integer index via binomial-coefficient lookups, in $O(d)$ either direction. Its `cofacetIterator/facetIterator` walk the *complete* `vertexCount`-point abstract simplex with no notion of `maxDimension` truncation at all — see [Hard-won invariants](#) before building anything new directly on them.

Cubical complexes

`Cube` (`cells/Cubical.scala`) is an elementary cube: a product of n (fixed ambient dimension) factors, each a degenerate point $[a, a]$ or a unit interval $[a, a+1]$, via the standard "doubled coordinate" encoding (axis k is $2a$ or $2a+1$). `opaque type Cube = Vector[Int] — Vector`, not `Array`, so structural equality works for `Chain`'s pivot tables. Boundary sign alternates over a non-degenerate axis's *rank among the other*

non-degenerate axes, not its raw coordinate position — get this wrong and $d(d(x))$ fails to vanish as soon as a cube has an interleaved degenerate axis, invisibly over a field of characteristic 2.

`CubicalGridStream` (`streams/CubicalStream.scala`) is a dense cubical complex over a full rectangular grid (the standard T-construction): a caller-supplied `topCellValue` gives every top-dimensional cube its own value, and every lower-dimensional cube's value is the `min` over the top cells containing it — computed directly via the cartesian product over the cube's degenerate axes rather than a recursive coface walk. `CubicalImage` builds these from images/voxel grids (`fromFlatArray/fromBufferedImage/fromFile/fromVoxelGrid3D`); sublevel vs. superlevel filtration is handled by negating values on load, not as a flag on the stream itself. `CubicalHomologyContext` is a one-line Cube-specialized wrapper around `CellularHomologyContext` — cubical complexes needed no new engine code, only a new `OrderedCell` instance.

Simplicial sets

`SSetElement[G](word, generator)` (`algebra/SSetElement.scala`) plus `FiniteSimplicialSet[G]` (`cells/SimplicialSet.scala`) implement finite simplicial sets in the classical Eilenberg-Zilber presentation: a finite set of non-degenerate generators per dimension, plus primitive face data `faces: G => IndexedSeq[SSetElement[G]]`. `word` is the degeneracy indices in normal form — **strictly decreasing**, not increasing, a direct consequence of the simplicial identity $s_i s_j = s_{j+1} s_i$ ($i \leq j$). `insertOuter/faceOf` implement the full operator algebra (s_i/d_i on arbitrary, possibly degenerate elements), which is what lets `FiniteSimplicialSet.validate()` check that hand-supplied face data actually satisfies the simplicial identities.

`FiniteSimplicialSet`'s companion object (`cells/SimplicialSet.scala`; the ordering helpers it uses live in `cells/SimplicialSetConstructions.scala`) builds new simplicial sets from old: `product/coproduct` (categorical product/coproduct — a product's non-degenerate simplices are pairs (a, b) with *disjoint* degeneracy words, not Eilenberg-Zilber shuffles, and its top dimension is $\maxDim(x) + \maxDim(y)$), and `quotient/identify` (attaching maps — `quotient` takes $G \Rightarrow \text{SSetElement}[G]$ rather than $G \Rightarrow G$ specifically so a cell can collapse down a dimension onto a degenerate point, the $\#$ -complex model of $\#P^2$ needs exactly this for one of a triangle's three edges).

`streams/SimplicialSetStream.scala` adapts a `FiniteSimplicialSet[G]` to `CellStream` for ordinary, unfiltered homology (every generator at filtration value 0); `fromStream` builds a `FiniteSimplicialSet` from any simplex stream. `streams/FilteredSimplicialSetStream.scala` is the genuine `StratifiedCellStream[G, Double]`, with a caller-supplied `filtrationValue` and `validateMonotoneFiltration` to check the one precondition every engine needs (a face's value never exceeds its coface's).

Cech complexes

`streams/CechStream.scala` (`CechCofaceSimplexStream/CechFiltration`) builds Cech complexes over `Simplex[Int]`, reusing `RipserCofaceSimplexStream`'s generic coface-generation loop via its `filtrationValueOverride` hook rather than a from-scratch algorithm. The geometric primitive is minimum-enclosing-ball radius (`com.dreizak:miniball`), not the Delaunay/QP machinery alpha shapes use — Cech membership depends only on a simplex's own vertices, not on every other point in the cloud. `CechFiltration` caches each simplex's radius and clamps it to the max of its own facets' cached radii, since a raw `Miniball` call can violate monotonicity by a floating-point ULP on near-degenerate input.

New-VR's Table-Lookup optimization and the packed Ripser engine's optimizations do **not** carry over to Cech: both are proven specifically for flag complexes / the max-pairwise-distance functional, and Cech is neither a flag complex nor governed by that functional. Cech complexes work with either generic engine (`CellularHomologyContext/CellularPersistenceInChunksContext`), cross-validated against each other; the packed and reference Ripser engines (specialized to the Vietoris-Rips functional) are not offered for it.

Witness complexes

`streams/WitnessStream.scala` implements De Silva & Carlsson's witness complex (2004), checked directly against JavaPlex's own `LazyWitnessStream/WitnessStream` Java source (not just the paper's prose). `LandmarkSelector.maxmin/.random` pick a landmark subset of a `FiniteMetricSpace[Int]`; `WitnessGeometry` precomputes the landmark-to-witness distance matrix `D` and each witness's sorted landmark-distance row (`mDim(k, witness)`), the $(k+1)$ -th nearest landmark — the one primitive both variants below are built from).

Two independent implementations, both reusing `RipserCofaceSimplexStream`'s generic coface-generation loop (the same reuse `CechCofaceSimplexStream` makes) rather than a from-scratch enumeration:

- `LazyWitnessSimplexStream` (JavaPlex's `LazyWitnessStream`): the lazy witness complex IS, by definition, the flag/cliue complex of a weighted graph, so `WitnessMetricSpace` reifies that graph as an ordinary `FiniteMetricSpace[Int]` (`distance(a,b)` = the edge witness value at a fixed `nu` in $\{0,1,2\}$) and hands it to `RipserCofaceSimplexStream` **unmodified** — no `filtrationValueOverride` at all; the inherited "max pairwise distance" flag extension is exactly what a lazy witness complex wants. Because it really is a flag complex, `PackedRipserCohomologyContext` — proven only for genuine VR diameters — is *also* valid here (any `FiniteMetricSpace[Int]`'s own diameter, not something VR-specific despite the class's name), cross-validated directly in `WitnessStreamSpec` rather than assumed.

- `WitnessCofaceSimplexStream` (JavaPlex's plain `WitnessStream`): NOT a flag complex — a k -dimensional simplex's own witness value uses a dimension-specific threshold (`m_k`, JavaPlex's own per-dimension array) that need not be monotone facet-to-coface on its own. `filtrationValueOverride` here is a genuinely recursive function (`max(own_k(sigma), max over sigma's own facets)`), `TrieMap`-memoized, since the base class doesn't memoize a caller-supplied override) — that recursive max is what makes "the complex at threshold R " automatically downward-closed for every R , the same way VR's max-pairwise-distance does, and it's also what makes JavaPlex's separate `containsElement(face)` gate redundant here (proof and empirical check both in `WitnessStreamSpec`). `maxFiltrationValue` defaults to `+Infinity`, not the metric space's enclosing radius — that shortcut is only valid for flag complexes.

A fact used to cross-validate the two constructions against each other (`WitnessStreamSpec`): the general complex's own 1-skeleton is identical to the lazy complex's at $\text{nu} = 2$ — both use the 2nd-nearest-landmark threshold for edges, just reached via different code paths.

Sheehy's sparse/approximate Vietoris-Rips filtration

`streams/SheehyRipsStream.scala` (`SheehyRipsSimplexStream`) implements Cavanna, Jahanseir & Sheehy's greedy- permutation reformulation (arXiv:1506.03797) of D.R. Sheehy's original net-tree construction (arXiv:1203.6786, DCG 49(4) 2013) — the two papers' own `epsilon` parameters are not comparable. Given a sparsity parameter `epsilon` in $(0,1)$ and a full greedy permutation of the point set (`LandmarkSelector.maxmin` run to `numLandmarks = metricSpace.size`, extended to also expose each point's own insertion radius `lambda`), the resulting persistence barcode is a $(1+\epsilon)$ -multiplicative approximation to plain Vietoris-Rips's own barcode, from a complex whose size is linear in n for point sets of bounded doubling dimension — fewer simplices to *reduce* at the same scale range, in exchange for a controlled, quantified loss of precision.

Like Cech/Witness above, this reuses `RipserCofaceSimplexStream`'s generic coface-generation loop rather than a from-scratch enumeration, but via a different shape: the ambient metric space is passed through `UNMODIFIED` (used only for combinatorial enumeration, never its own `.distance`), and one `filtrationValueOverride` computes every dimension ≥ 1 directly from the greedy permutation's own `lambda` values — not a reified weighted `FiniteMetricSpace` the way `WitnessMetricSpace/DtmRipsSimplexStream`'s own reified space are. The reason is a genuine exclusion rule with no pairwise-only expression: a k -simplex's value is the max of its own edges' births, but ONLY if that value doesn't exceed the smallest of its OWN VERTICES' "vanish" times $(\text{lambda}_p * (1+\epsilon)^2 / \epsilon)$ — a condition that has to see every vertex of the simplex at once, not just a pair, so a plain "flag complex over a weighted pairwise distance" shape can't carry it.

A documented, verified gap in the source paper's own published algorithm: its `EdgeBirthTime` (Algorithm 3) computes an edge's birth from its two endpoints alone, with no check against either one's own vanish time — but the paper's own general `SimplexBirthTime` rule (Section 5.3) requires exactly that check, and an edge is simply that rule's $k=1$ case, not a special one. Verified with two counterexamples, the second checked directly against the paper's own restricted neighbor-search bound (not just this implementation's own all-pairs enumeration) — see `.claude/WORKLOG-sheehy-rips.md` for both. This implementation applies the check uniformly from dimension 1 up.

This construction is deliberately $O(n^2)$ (every pairwise edge birth materialized directly), not the paper's own $O(n \log n)$ neighbor-search algorithm (Section 5, Algorithms 1-4, not implemented) — the payoff is a smaller complex to reduce, not a faster one to build, the same honest framing as Cech/Witness/ α above. Refuses `engine="ripser"`: a simplex's value here is not simply the maximum ambient pairwise distance among its vertices (some pairs are sparsified to a smaller value, others excluded outright), so `PackedRipserCohomologyContext`'s `insertionDiameter/apparent-pairs` optimizations — proven specifically for that functional — do not apply. `naive/chunks/cohomology` all consume it like any other `CofaceSimplexStream[Int, Double]`; `chunks` is cross-validated fresh against `naive` (`SheehyRipsStreamSpec`), not assumed to carry over from any other construction — see [Persistence engines](#)'s own `streams-vs-engines` table for the full picture across every construction, not just this one.

Metric spaces

`FiniteMetricSpace.scala` abstracts "distance + finite point set": `ExplicitMetricSpace` (raw distance matrix), `EuclideanMetricSpace` (coordinate array, on-demand Euclidean distance, VP-tree-backed `neighbors` query), `IntMetricSpace` (reindexes to contiguous `0 until size`), `SparseMetricSpace` (reports `+Infinity` beyond a fixed diameter cutoff, bounding Vietoris-Rips construction to a finite neighborhood per point).

Opt-in parallelism

A few of the more expensive per-cell computations can run on the common `ForkJoinPool`, opt-in via a constructor flag defaulting to `false`, with output that is deterministic either way: `AlphaDQPSettings.parallel` (the per-vertex QP solve), `CubicalGridStream.parallelFiltrationValue`, and `CechCofaceSimplexStream.parallelFiltrationValue`. These are worth reaching for on large inputs where the per-cell cost is real (a QP solve, a Miniball radius) but shouldn't be assumed to give a large win — measured speedups range from a few percent (cubical, Cech — the per-cell cost there is small enough that scheduling overhead eats most of the gain) up to roughly 2-4x for the alpha-complex QP solve at a few hundred points and above, where each per-vertex solve is genuinely expensive.

Barcode.scala: representing the output

`org.appliedtopology.tda4j.barcode` defines `BarcodeEndpoint` (`PositiveInfinity/NegativeInfinity/ OpenEndpoint/ClosedEndpoint`, with a total order that correctly interleaves finite and infinite endpoints) and `PersistenceBar[FiltrationT, AnnotationT]` (dimension, lower/upper endpoint, an optional annotation — in practice always the representative `Chain`). `Barcode` additionally implements algebra on finitely-presented persistence modules: `image/kernel/cokernel` of a map between two barcodes represented as a matrix — useful for interleaving distances or persistence-module morphisms, not needed for ordinary persistent-homology computation.

package.scala: TDAContext

```
class TDAContext[VertexT: Ordering, CoefficientT: Field, FiltrationT: Ordering]
  extends SimplicialHomologyContext[VertexT, CoefficientT, FiltrationT]():
  val chainIsRingModule
  = summon[Chain[SimplicialSimplex[VertexT], CoefficientT] is RingModule { type R
  = CoefficientT }]
  export chainIsRingModule.*
  given [T: Ordering]
  => Conversion[SimplicialSimplex[T], Chain[SimplicialSimplex[T], CoefficientT]] = Chain.apply
```

`TDAContext` takes **three** type parameters (`VertexT`, `CoefficientT`, `FiltrationT`). It is a `SimplicialHomologyContext` (see **Persistence engines**), plus it exports chain-arithmetic operators (`+`, `-`, `#`, ...) into your namespace and provides an implicit `SimplicialSimplex -> Chain` conversion so you can write `#(1,2)` - `#(2,3)` directly — the basis for the **User's Guide's** Scala quick-start.

Persistence engines: what to trust, and why

`homology/Homology.scala`, `homology/PackedRipserCohomology.scala`, and `homology/Cohomology.scala` contain **four independently-implemented algorithms across five concrete classes**. They share the Chain reduction primitives from **Architecture**, but they are not variants of one shared engine — a fix or bug found in one does not imply anything about the others. Read this page before choosing which engine to build on.

Three of the five (`CellularHomologyContext`, `CellularPersistenceInChunksContext`, `CellularCohomologyContext`) additionally implement the common `homology.PersistenceEngine[CellT, C] trait (def barcode(stream): List[PersistenceBar[Double, Chain[CellT, C]])` via `PersistenceEngine.naive/.chunks/.cohomology` factories — a thin, opt-in adapter over each engine's own incremental API, for a caller (the MATLAB/CLI facade) that just wants a finished barcode without hand-writing each engine's own construct/advance/read dance. It does not change any engine's own contract; the incremental API (`advanceTo/diagramAt/barcodeAt`) stays available on the concrete class. `PackedRipserCohomologyContext/RipserCohomologyContext` don't implement it — they consume a `FiniteMetricSpace[Int]` directly, not a stream (see engine 4 below).

1. CellularHomologyContext / SimplicialHomologyContext — reference-grade, generic

`CellularHomologyContext[CellT: OrderedCell, CoefficientT: Field, FiltrationT: Ordering]` is the naive single-pivot-table boundary-reduction algorithm: process cells in filtration order, reduce each cell's boundary against pivots recorded so far via `Chain.reduceBy`, and the cell either opens a class (reduced boundary is zero) or closes one. No clearing, no chunking, no cohomology/twist optimization. Generic over *any* `CellT: OrderedCell` — this is what makes it the engine `Cube` and `FiniteSimplicialSet` slot into with no new engine code (`CubicalHomologyContext`, `SimplicialHomologyContext` are one-line specializations of it). It's the **oracle every other engine here gets cross-validated against**.

It's also the only engine with genuine incremental querying: `HomologyState.advanceOne()/advanceTo(f)/advanceAll()`, and `diagramAt(f)/barcodeAt(f)` can be called mid-stream to get the diagram *as of* filtration value `f`. `barcodeAt` annotates every bar with a real representative cycle, tracked via a parallel V-column alongside the ordinary reduction. `TDAContext` (root `package.scala`) extends this class.

2. CellularPersistenceInChunksContext / PersistenceInChunksContext — chunked, generic

`CellularPersistenceInChunksContext[CellT: OrderedCell, CoefficientT: Field](maxDim: Int = 5)` implements the parallelizable "clear-and-compress" algorithm: local reduction per chunk, active-entry marking, then global column compression/reduction. `PersistenceInChunksContext[VertexT, CoefficientT]` is a one-line `Simplex`-specialized subclass, mirroring `SimplicialHomologyContext`'s relationship to engine 1 — the class has no `Simplex`-specific behavior anywhere in its body, so any `OrderedCell` slots in directly.

Dimensions 0 and 1 go through a dedicated union-find fast path (`unionFindDim01`) instead of the general Chain-based reduction, since both dimensions are mathematically forced to agree with the general machinery on a fixed total order — a real, measured win when a large point cloud's dimension-0/1 structure dominates the complex. `barcodeAt(f)` returns a real representative for **every** bar, at every dimension, including essential classes — reconstructed incrementally from this class's own already-computed boundaries/cleared/paired/killer state (`vclof`, memoized) rather than by delegating to a second engine run. `advanceAll()` runs the whole pipeline in one shot; there is no incremental querying the way engine 1 has.

3. RipserCohomologyContext — test/reference oracle for engine 4

Persistent cohomology via Ulrich Bauer's Ripser algorithm (arXiv:1908.02518), specialized to `Simplex[Int]` Vietoris-Rips/cliue complexes via `SimplexIndexing`'s combinatorial number system — a deliberate narrowing from the generic `CellT: OrderedCell` engines above. One-shot only (`persistentCohomology()`, no incremental querying).

This class is not what production code calls. `PackedRipserCohomologyContext` (engine 4) is a faithful re-keying of the same algorithm onto a packed representation, and is what `matlab.TDA4j`'s `engine="riper"` actually uses. This class's remaining value is narrower than "an independent check on the Ripser algorithm": both classes share `SimplexIndexing`, so a bug there passes both silently (engine 1 is the actually-independent oracle for the algorithm itself). What this class *does* catch is anything specific to engine 4's own packed representation — its `DiameterIndex` carrier's index-only `equals/hashCode`, its index-keyed lookup maps — that no other spec would flag. Its `Simplex[Int]`-keyed chains are also more directly legible for hand-debugging. Kept fully maintained; don't add new production call sites against it.

Structural points worth knowing:

- **Clearing is required for correctness, not an optional speedup:** a simplex already claimed as a pivot one dimension down must be excluded from the next dimension's essential-index count entirely, not merely checked for "does its own column reduce to zero" — Definition 3.2/Proposition 3.1 of the paper.

- **Apparent pairs (Definition 3.2/Proposition 3.9) are partially implemented:** a genuine mutual apparent-pair check identifies the pivot directly and skips `Chain.reduceBy`'s reduction pass for it (a measured 1.35x-1.8x win, growing with complex size). What is *not* implemented is Ripser's further optimization of never building an apparent simplex's coboundary at all — `coboundaryOf(sigma)` is still called in full on the shortcut path, since `basis(tau)` needs the complete reduced column, not a truncated stand-in. Getting the fully lazy version would also require restructuring how each dimension's candidate simplices are enumerated in the first place (currently eager, unlike Ripser's own incrementally-assembled `columns_to_reduce`) — a larger, separate project.

4. PackedRipserCohomologyContext — the production Ripser engine

Same algorithm as engine 3, method for method, keyed on a packed (`Double`, `Long`) diameter/combinatorial- index pair (`DiameterIndex`) instead of a materialized `Simplex[Int]`. This is what `matlab.TDA4j's engine="riper"` calls, and the fastest, most memory-efficient engine in the library — a deliberate representation choice on top of an already-validated algorithm, not a new algorithm. Kept in its own file, separate from the three generic-`OrderedCell` algorithms in `Homology.scala`, since it's specific to Vietoris-Rips/`SimplexIndexing` rather than a general engine.

`DiameterIndex` overrides `equals/hashCode` to consider only the combinatorial index, not the diameter — so "same simplex" is true by construction regardless of which floating-point path computed its diameter, sidestepping a real footgun (two carriers for the same simplex comparing unequal on floating-point noise).

5. CellularCohomologyContext — generic cohomology, for every cell type

Persistent cohomology, generic over `CellT`: `OrderedCell` — the cohomology counterpart to engine 1, filling in what used to be a real asymmetry: cohomology in this codebase meant engines 3/4 only, both hardcoded to `Simplex[Int]`. Cube, `FiniteSimplicialSet` generators, and `Simplex[Int]` complexes that aren't Vietoris-Rips (Cech, Alpha, the general witness complex) had no cohomology option at all before this class — not even Cech/Alpha, despite sharing `Simplex[Int]` as a cell type, since engines 3/4's speed optimizations (`insertionDiameter`, apparent pairs) are proven specifically for the max-pairwise-distance functional, not Cech's circumradius, Alpha's own filtration, or the general witness complex's per-dimension threshold. (The *lazy* witness complex is the one exception among these: it genuinely IS a max-pairwise-distance flag complex under its own `WitnessMetricSpace`, so engines 3/4 both work for it directly — see `architecture.md`'s "Witness complexes" section.)

The key idea: the coboundary matrix persistent cohomology reduces is the transpose of the ordinary boundary matrix, same coefficients. Every stream this class targets already gets fully materialized before persistence runs (unlike Vietoris-Rips at the scale engines 3/4 target), so this class builds the coboundary relation directly by inverting each materialized cell's own `boundary[CoefficientT]` call, one dimension

band at a time (built, used, and discarded before moving to the next dimension, bounding peak memory to the largest single band) — no `SimplexIndexing`-style combinatorial machinery, and no per-cell-type coboundary formula.

No `maxDim` parameter, and no apparent pairs — both deliberate. This class computes cohomology up to whatever top dimension the materialized stream actually contains; a caller wanting only $H_0 \dots H_k$ truncates the *input stream* first (`LimitedCofaceSimplexStream(stream, k + 1)`, the same mechanism engine 3's own test suite and `engine="naive"` already use) and drops `dim == k + 1` bars from the result afterward — deleting a whole footgun class (engines 2, 3, and 4 each had to fix a "maxDim means top built vs. top reported degree" bug once) rather than reimplementing it a fifth time. Apparent pairs' entire point is avoiding coboundary *enumeration* — this class has none to avoid, since it must materialize the coboundary relation for every cell up front just to have "coboundary" exist at all; porting the mutual-pair check would save a basis write and one already-cheap `Chain.reduceBy` call, noise-level and not worth the machinery. See `.claude/DESIGN-generic-cohomology.md` for the full derivation of both calls.

Representatives: every bar carries a V-column, the same way engines 3/4 already track one. Only an *essential* bar's V-column is a genuine cocycle (`d(vcol) == 0`) by construction — a finite bar's V-column has coboundary equal to its own nonzero reduced pivot chain instead (still a valid representative on the bar's own living interval, just not a cocycle over the whole complex). `coboundaryOfChain` exists specifically to check this for essential bars. This is also the class's actual point, not an afterthought: over a field the cohomology barcode is identical to the homology barcode, so a bars-only version would be entirely redundant with engine 1, which already covers every cell type this class does.

There is no `Cocell/OrderedCocell` typeclass backing this (an earlier, unimplemented, dual-to-`Cell` trait pair was removed outright while building this class) — coboundary is *extrinsic* to a cell (it depends on which higher-dimensional cells exist in the ambient complex), not intrinsic the way *boundary* is, so a per-cell coboundary method with no complex to consult was never the right shape.

Streams × engines: what works with what

Every complex construction in this codebase produces a `CofaceSimplexStream/CellStream` that, in principle, any of `matlab.TDA4j`'s four engine values could consume — but two of those engines (`PackedRipserCohomologyContext/engine="riper"` and `CellularPersistenceInChunksContext/engine="chunks"`) were built against specific assumptions (a genuine flag complex, a max-pairwise-distance filtration functional, vertices born at filtration 0) that not every construction satisfies, and refusing the combination outright (a clear `IllegalArgumentException`, not a silently wrong barcode) is deliberate. This table is generated by reading `matlab.TDA4j`'s own `dispatch/resolveWitnessEngine/dispatchCubical` — the single source of truth for every refusal and default — not by inference from a construction's own doc; re-check that source if this table and the code ever disagree.

Complex (complex=)	Default engine	ripser	naive	chunks	cohomology
vr	ripser	yes	yes	yes	yes
cech	naive	no — not a max-pairwise- distance functional (radius, not diameter)	yes	yes	yes
witness, witnessVariant	ripser	yes — genuinely a flag complex under its own reified WitnessMetric	yes	yes	yes
witness, witnessVariant=general	naive	no — not a flag complex (dimension- specific threshold)	yes	no	yes
dtm-rips	naive	no — vertices aren't born at 0, and the weighted- Rips functional isn't insertionDiam compatible	yes	yes	yes
dtm-alpha	naive	no — no notion of a Vietoris- Rips complex at all	yes	no — shares AlphaComplexDQP's known stall/ OOM risk (see alpha- complex.md)	yes
alpha	naive	no — no notion of a Vietoris-	yes	no — known stall/OOM risk	yes

Complex (complex=)	Default engine	ripser	naive	chunks	cohomology
		Rips complex at all		(HomologySpec' BarcodeRegres	
sheehy-rips	naive	no — a yes simplex's value is not the maximum ambient pairwise distance among its vertices (some pairs sparsified away, others excluded outright)		yes	yes
cubical (computeFromCi no complex key)	naive	no — yes PackedRipserC is specialized to Simplex[Int]		yes	yes
simplicial sets	(no matlab/cli entry point at all — construct SimplicialSetStream/FilteredSimplicialSetStream and drive any generic engine directly)				

Reading the "no" cells as one-line reasons, grouped by root cause:

- **Not a flag complex** (cech, witness/general): a k-simplex's value isn't determined by its own edges' values alone, so `insertionDiameter`'s incremental recurrence has nothing valid to incrementally update. `sheehy-rips` belongs here too, not with `dtm-rips` below, despite looking pairwise-derived at a glance: its own `filtrationValueOverride` needs to see every vertex of a k-simplex at once (the

min-over-vertices vanish exclusion check), not just its edges — proven by construction, not merely asserted: a hand-derived fixture (`SheehyRipsStreamSpec`) exhibits a triangle whose three edges are ALL individually present and finite, yet the triangle itself never appears — the one thing an actual flag complex can never do.

- **Not a Vietoris-Rips complex at all (alpha, dtm-alpha):** `PackedRipserCohomologyContext` consumes a `FiniteMetricSpace[Int]` directly and enumerates cliques via `SimplexIndexing` — there is no Delaunay/power- cell structure it could route through instead. These two also refuse chunks, but for a third, unrelated reason: a known stall/out-of-memory risk in `AlphaComplexDQP` at scale, not an algorithmic mismatch (see `alpha-complex.md`; `HomologySpec`'s `BarcodeRegressionSpec` stays `skipAll`'d for the same reason and is the regression pin, not a live check).
- **A genuine flag complex, but vertices aren't born at 0 and the edge functional isn't plain max-pairwise-distance (dtm-rips only):** `PackedRipserCohomologyContext`'s two production-critical optimizations (`insertionDiameter`, `apparent pairs`) are proven specifically for `MaximumDistanceFiltrationValue` on the metric space handed to it — a weighted filtration value invalidates both proofs even though the complex itself is, combinatorially, an ordinary flag/cliique complex (unlike `sheehy-rips` above).
- **Representation-specific (cubical):** `PackedRipserCohomologyContext`/`RipserCohomologyContext` are hardcoded to `Simplex[Int]`'s combinatorial-number-system indexing (`SimplexIndexing`); `Cube` has no equivalent encoding built for it (a `CubicalRipser`-style dedicated fast engine remains a documented future direction, `DESIGN-fast-cubical-engine.md`, not something engine 3/4 already do).

`engine="cohomology"` (`CellularCohomologyContext`, engine 5 above) is the one column with no "no" cells for a reason: it's generic over `CellT: OrderedCell` with no per-construction speed assumptions baked in, at the cost of none of engines 3/4's Vietoris-Rips-specific optimizations — see engine 5's own section above for what that tradeoff actually buys and costs.

Choosing an engine

Need	Engine
Exploration, intermediate-filtration queries, representative cycles, any <code>OrderedCell</code> type	<code>CellularHomologyContext</code> / <code>TDAContext</code>
Large complex, chunked/parallelizable, representatives for every bar including essential ones	<code>CellularPersistenceInChunksContext</code>

Need	Engine
Fast, memory-efficient cohomology on a Vietoris-Rips/cliique complex over integer vertex labels	<code>PackedRipserCohomologyContext</code> (what <code>engine="ripser"</code> uses)
A <code>Simplex[Int]</code> -keyed reference implementation for hand-debugging engine 4	<code>RipserCohomologyContext</code> (test oracle, not a production choice)
Cohomology (real cocycle representatives) on <code>Cube/FiniteSimplicialSet/Cech/Alpha/</code> general witness complex, or any <code>OrderedCell</code> type engines 3/4 can't serve	<code>CellularCohomologyContext</code> (what <code>engine="cohomology"</code> uses)

Hard-won invariants you must not break

These are the load-bearing, non-obvious rules this codebase depends on. Every one of them was found by a real bug, not by reading a spec — which is itself the pattern worth internalizing (see **Process: validate empirically, not by inspection** at the bottom). If you're extending the persistence engines, complex-construction streams, or anything Ripser-flavored, read this page in full before you start.

1. given/implicit resolution happens once, at construction, not per call

A `given [CellT: Ordering, CoefficientT: Field] => (Chain[CellT, CoefficientT] is RingModule)` instance resolves its `Ordering[CellT]` context-bound parameter **exactly once**, when the instance is first summoned — every method built on it permanently closes over that one resolution. Scala's implicit search is static (compile-time, lexical), not dynamic; calling those methods later, from a scope where a *different* `Ordering[CellT]` given is available, does not re-resolve anything.

Concretely: `Simplex[VertexT]` is `OrderedCell`'s own `.ordering` is a fixed lexicographic order, and `Chain.scala`'s `given [CellT: OrderedCell] => Ordering[CellT] = oCell.ordering` fallback means any `chainRM = summon[Chain[Simplex[VertexT], CoefficientT] is RingModule]` written *before* a stream-specific `given Ordering[Simplex[VertexT]] = stream.filtrationOrdering` is in scope silently captures that lexicographic fallback instead — every chain built through `#/.scale` afterward pivots on vertex label order, not filtration order, regardless of what's nominally in scope at the call site.

The fix pattern, everywhere it's applied: declare `given Ordering[CellT] = stream.filtrationOrdering` as the *first* statement inside whatever state object holds the stream, before summoning `chainRM`.

This is genuinely dangerous rather than a style nit because it compiles cleanly either way and only manifests as a wrong *answer* on inputs where lexicographic and filtration order actually disagree — most hand-built fixtures don't exercise that unless someone deliberately constructs one (a fixture with vertex 1 born early and vertex 9 born late is the minimal discriminator). **Not every class-scope `chainRM` instance is a live bug**: whether it matters depends on whether anything downstream trusts that instance's output directly for pivot identity, or whether every consumer re-resolves ordering itself (e.g. via `Chain.reduceByUntil`, a `def` that resolves fresh at each call site). This needs a case-by-case, empirically-verified audit every time, not a blanket rule.

2. A stream's iteration order and its pivot order must be the *same* total order

Boundary-matrix reduction requires columns (processing order) and rows (pivot order) to be indexed by one shared total order. Concretely: a stream's `.iterator/iterateDimension` order and its

`filtrationOrdering` (used to key `Chain.reduceBy`'s map) must be *the same ordering*, one the consistent `.reverse` of the other — not merely "each independently a valid total order." Two orderings that are each individually consistent but disagree on which of two *tied* cells comes first corrupt reduction in a way that's easy to miss on random test input, because Vietoris-Rips complexes have *structural* ties (every simplex of dimension ≥ 2 ties with its own longest edge) while random fixtures often don't — see **Degeneracies**.

Symptom: `IllegalStateException: reduction pivot ... was not a recorded open class`. This means a cell got selected as a pivot the algorithm's own invariants say should have been impossible — check whether iteration order and pivot order for the stream in play are provably the same ordering before looking in the reduction code itself.

`.reverse` on the *same* `Ordering` object is safe; building a *second*, independently-written "reversed" comparator is not, even when it looks equivalent on paper — a hand-written second comparator's tie-break direction doesn't automatically stay consistent with a separately-reversed primary key, and this has broken more than one stream in this codebase's history for exactly that reason.

3. Colex vs. lex tie-breaks are not interchangeable in Ripser-flavored code

The generic default tie-breaks on plain lexicographic vertex-set order. `EnumeratingCofaceSimplexStream`'s and `RipserCohomologyContext`'s own orderings deliberately use **colexicographic** order instead, via `SimplexIndexing`'s own combinatorial-number-system index — the exact tie-break Ripser's Definition 3.2/Proposition 3.9 (apparent pairs) are stated in terms of. Don't "simplify" a colex ordering to plain lex in code that touches `SimplexIndexing`/the Ripser engines; they need to agree with each other, not just each independently be a valid tie-break.

4. `Chain.reduceBy/reduceByUntil`, never hand-rolled `Chain` arithmetic, inside a reduction loop

`Chain`'s `+/-/#` operators only lazily collapse the *head* of the underlying `PriorityQueue`, so a hand-rolled fold that repeatedly subtracts terms builds an ever-growing backlog of uncanceled duplicate entries — confirmed directly by a first draft that hung on an 8-12 point complex that should take milliseconds. `Chain.reduceBy/reduceByUntil` go through a `SortedMap` that collapses duplicates on every insertion. Reserve raw chain arithmetic for small, one-shot combinations (building a V-column fold), never for anything that accumulates over many reduction steps.

5. Combinatorial helpers over the full point set don't know about `maxDimension` truncation

`SimplexIndexing.cofacetIterator/facetIterator` operate purely combinatorially over the complete n -point abstract simplex — no concept of any per-engine dimension cap.

`RipsCohomologyContext.coboundaryOf` truncates explicitly instead, which is what makes top-dimension simplices come out essential. Any new code built directly on `SimplexIndexing`'s iterators inherits none of that truncation and needs its own manual guard at the call site — `SimplexIndexing` itself has no way to know what any particular caller's cap is. The symptom (wrong finite bars where essential ones were expected) looks like a reduction bug, not an off-by-scope error two layers away, so it's easy to misdiagnose.

6. `iterateDimension`'s domain must be contiguous from 0

`StratifiedCellStream.iterator`'s default implementation is `Iterator.from(0).takeWhile(iterateDimension.isDefinedAt).flatMap(iterateDimension)` — it stops at the first dimension the stream doesn't define, and never asks about anything past it. A stream whose domain has a *gap* (defined at 0 and 2 but not 1) silently truncates at the gap rather than skipping past it. Every concrete stream in this codebase satisfies this for a structural reason, not by convention alone: a simplicial or cubical complex can't have a d -cell without its $(d-1)$ -dimensional faces, so "no cells at d " implies "no cells beyond d " too. A hand-built or generated stream that doesn't have this structural guarantee needs to enforce contiguity itself — an unbounded case `d => ... catch-all` with no upper bound is also a hazard here (`isDefinedAt` always `true`, which defeats the "stops at the first gap" logic entirely and can loop until `Int` wraps around).

7. Opaque-type extension methods belong in the type's own companion object

An extension whose *receiver* is an opaque type (`Simplex`, `Cube`) should live inside that type's own companion object (`object Simplex/object Cube`), not as a top-level extension clause — extension-method resolution checks a receiver type's companion object by nominal type, so two unrelated opaque types can safely reuse the same method name this way (`Simplex.underlying/Cube.underlying` never collide). Two hazards to know before leaning on this: opaque-type transparency is scoped to the *whole file* the opaque type is declared in, so code in that same file calling the type's own extensions by dot-syntax can resolve to a same-named member of the underlying representation type instead of the intended extension — keep an opaque type's `OrderedCell`/similar instances in their own file when this matters. And a companion-object extension can still lose to a same-named `stdlib` extension reachable via a wildcard import at some call site (`min/max` deliberately stay top-level extensions on `Simplex` for exactly this reason, to avoid losing to `scala.math.Ordering.Implicits`).

Process: validate empirically, not by inspection

Every substantive design decision in this codebase's persistence-engine work (pivot orientation, whether clearing is optional, whether an ordering fix is complete, whether an optimization is sound) has gone

through at least one empirical discriminating test before being trusted — never a plausibility argument alone, even a careful one. "This class-scope `chainRM` should be inert because X" has been both correct and wrong for structurally similar-looking classes in this codebase; only a discriminating test told the two apart.

The pattern to carry into new work: **when a change to reduction/ordering logic seems obviously sound by inspection, build the cheap discriminating test anyway before implementing it for real.** A good discriminating fixture deliberately puts filtration order and some other natural order (vertex label, insertion order) in conflict — a fixture where they happen to agree will not catch this class of bug.

Degeneracy behaviors that look like bugs but aren't

A dedicated page so you don't "fix" correct-but-surprising behavior. Each of these has, at some point, looked like a bug to someone reading the output for the first time.

Alpha complexes in degenerate (cospherical) position are not Delaunay subcomplexes

In degenerate (cospherical) position, the alpha complex is **not** a subcomplex of some triangulation you could point to. k cospherical sites sharing a Voronoi vertex contribute a $(k-1)$ -simplex — a unit grid in the plane produces 3-simplices (one per unit square, since the four corners of each square are cospherical), not just triangles from an arbitrary triangulation choice; a unit grid in $\mathbb{R}^3$ produces 7-simplices (one per unit cube). **Truncating at the ambient dimension gives the wrong homotopy type, not merely a truncated one.** Users coming from CGAL or GUDHI will not expect this, because those tools typically report a triangulation (a choice among several valid ones), not the actual alpha complex in the degenerate case — but it's correct here, not a bug. AlphaValidationSpec's grid test exercises a mild version of this directly: two near-collinear rows produce two "sliver" simplices with circumradius roughly 40-60x the point cloud's diameter, and both the DQP and Helix backends correctly include them. See [Alpha complex: DQP vs Helix](#) for the developer-facing detail on both backends.

Zero-persistence (zero-length) bars are real output, not noise to filter

RipserCohomologyContext emits these deliberately — e.g. an edge tied in filtration value with the triangle that immediately kills it. Definition 3.2/Proposition 3.9's *apparent pairs* (see [Persistence engines](#)) are exactly the zero-persistence pairs — dropping them silently at the engine level would be wrong at this stage of the pipeline, even though a downstream visualization might reasonably choose to filter them before display. If you see a $[3.0, 3.0)$ -style bar in test output, that's not evidence of a bug on its own.

A Vietoris-Rips complex with $\text{maxDimension} \geq 2$ always has ties

Every simplex of dimension ≥ 2 ties in filtration value with its own longest edge, by construction (a simplex's filtration value — maximum pairwise distance among its vertices — is always realized by some edge face of it). This is the *normal* case for VR complexes, not evidence of a degenerate or adversarial input, and any reduction code path that implicitly assumes "ties are rare" will misbehave on completely ordinary data. This is precisely why [Hard-won invariants #2 and #3](#) about tie-break consistency exist and matter in practice, not just in adversarially constructed test cases.

A quotient simplicial set can collapse a cell to a *degenerate* point, not just merge it with a peer

quotient's attaching map is $G \Rightarrow \text{SSetElement}[G]$, not $G \Rightarrow G$, specifically because some quotients need a cell to crush down a dimension entirely rather than merge with a same-dimension peer — the standard $\#$ -complex model of $\#P^2$ glues two of a filled triangle's three edges into a loop, but the third collapses entirely onto a degenerate point over a vertex. If you're building a new attaching map and find yourself wanting to map a generator to a lower-dimensional target, that's expected, not a sign the API is being misused.

A cubical or Cech complex having "extra" simplices/cubes at a shared tie is not a bug

The same underlying fact as the alpha-complex case above shows up differently in other constructions: a cubical grid's dense T-construction cell count (`prod_i (2*shape(i)+1)`) is much larger than the pixel/voxel count itself — a 256x256 image is 263,169 cells, not 65,536 — because every lower-dimensional face of every pixel is its own cell, most of them shared between adjacent pixels. This is the correct cell count for the cubical complex, not evidence of a construction bug.

Alpha complex: DQP vs Helix

Two independent backends compute alpha complexes; `AlphaShapes(points, dispatch)` (alpha/`AlphaShapes.scala`) chooses between them. This page is the developer-facing view. For the user-facing framing (which one to pick, and the honest tradeoffs), see the [User's Guide](#).

Dispatch

```
object AlphaShapes:
  def apply(pts: Seq[Array[Double]], dispatch: String = "default")
    (using epsilon: Epsilon): AlphaShapes
```

`dispatch = "default"` always resolves to "helix" regardless of point-cloud shape — "DQP" must be requested explicitly. Both backends extend the common `AlphaShapes` abstract class, so they're dispatch-interchangeable as far as any code consuming the resulting stream is concerned — `AlphaComplexSpec` runs identical property checks against both to enforce this.

HelixDelaunay — an actual Delaunay triangulation

Builds an actual Delaunay triangulation incrementally: finds a bootstrap simplex, then walks the frontier of facets, testing candidate points against each facet's supporting hyperplane and circumsphere. `filtrationValue` returns the unsquared circumradius, matching DQP's own units after `radiusOf`.

Known, quantified limitation: on point clouds with a near-cospherical local cluster (competing candidate simplices' circumradii agreeing to 4-5 significant figures — closer than the tiling logic's own near-tie detection catches, since that logic only checks ties against one already-chosen candidate, not across competing candidates), the frontier walk's greedy search becomes order-dependent and can converge on a locally-consistent but globally wrong triangulation. Measured: zero failures across 20,000-trial fuzz sweeps at ambient dimension 2 and 5, but roughly 1-in-170 at ambient dimension 4 with 20-30 points — ordinary-looking inputs, not contrived counterexamples. A real fix needs joint near-tie detection across all competing candidates before committing to one; this is a genuine algorithm change, not a bounded bug fix.

Practical consequence: Helix is not a fully reliable ground truth for automated cross-validation fuzzing at ambient dimension ≥ 4 without an assurance against near-cospherical local structure. Broad `forall`-based DQP-vs-Helix comparisons in `AlphaCrossValidationSpec` are deliberately kept out of `sbt test` (available as manually-invoked diagnostic methods instead) — a real Helix failure would otherwise masquerade as a DQP regression or vice versa.

AlphaComplexDQP — dual active-set QP, never builds Delaunay at all

Implements Erik Carlsson & John Carlsson, *Computing the alpha complex using dual active set quadratic programming*, Scientific Reports 14:19824 (2024), <https://doi.org/10.1038/s41598-024-63971-3>. Instead of building the Delaunay complex and reading off which simplices survive, it answers a per-simplex *feasibility* query directly — "is this Voronoi/power face nonempty, and does it meet the ball of radius r ?" — posed as a convex QP and answered in the Lagrangian dual, which is what lets it scale to very high ambient dimension where Delaunay is infeasible.

The QP solver follows DAQP (Arnström, Bemporad & Axehill, IEEE TAC 67(8):4362-4369, 2022), collapsed to Cholesky update/downdate of the active working set (`CholeskyWorkspace`) since the paper's problem is already in DAQP's canonical least-distance inner form.

Math cheat sheet

Base vertex x , neighbours x_i , power weights p . **Filtration values are squared radii (powers)** per the paper's Definition 10 — `AlphaComplexDQP.radiusOf` takes the square root, and `AlphaShapeDQP.filtrationValue` goes through `radiusOf` (not the raw squared value) so it matches `HelixDelaunay.filtrationValue`'s units. The dual objective only needs *squared distances*: $B_{ij} = (d^2(i, x) + d^2(j, x) - d^2(i, j))/2$ — which is why `PowerDistance` sits on squared distance rather than extending `FiniteMetricSpace` directly, bridging via `PowerDistance.toMetricSpace` only where interop is actually needed (e.g. `cechNeighbours()`'s VP-tree spatial index). **Caveat:** B is PSD only for Euclidean-embeddable metrics.

`AlphaShapeDQP` always computes the complete, **untruncated** alpha complex, matching `HelixDelaunay`'s own always-untruncated behavior (a finite radius bound would silently exclude the arbitrarily-large-circumradius simplices degenerate configurations legitimately produce — see **Degeneracies**). Callers who want an actually radius-truncated alpha complex should call `AlphaComplexDQP.euclidean(points, maxRadius, maxDimension, settings)` directly.

Numerical-robustness decisions — settled, not casually retunable

- **rankTolerance default is 1e-6, not the more "obvious" 1e-12.** A Schur-complement ratio as large as $\sim 1e-8$ can poison the Cholesky factor and cause genuine non-terminating active-set cycling; $[1e-7, 1e-5]$ is the empirically-verified safe range, and $1e-4$ starts silently rejecting genuinely non-degenerate directions.
- **Ratio-test ties break by the constraint's global index**, not its position in the working set (Bland's rule anti-cycling) — working-set position isn't a stable ordering.
- **Known, accepted limitation:** when the entering variable's Schur complement is small and no active inequality can be swapped out to compensate, `DualQP.solve` conservatively treats the candidate as

infeasible rather than committing the small pivot directly — this can wrongly exclude a genuinely-Delaunay simplex near certain near-degenerate configurations. A "more correct" fix (commit anyway when mathematically justified) was tried and reverted: two near-identical Schur-complement ratios required opposite handling in practice, showing no fixed numerical threshold can safely distinguish the two cases from that ratio alone — it depends on the whole working set's conditioning. `solveAtVertex` has a defense-in-depth per-candidate catch so an uncharacterized non-convergence case excludes just that one candidate rather than aborting the whole complex.

- **Vertex filtration values must go through `space.weight(x)`, not a bare `0.0` default** — only correct in the unweighted case; with nonzero weights a `0.0` default silently breaks the monotonicity invariant that a vertex's filtration value must not exceed an edge's through it.

`AlphaComplexDQPRegressionSpec` pins hand-verified adversarial point clouds (the two cases above) as permanent regression tests, alongside a broader property suite (`minTestsOk = 2000`, not `scalacheck`'s default of 100 — the bugs above had verified failure rates as low as 1-in-12000). `AlphaComplexDQPSpatialIndexSpec` cross-checks the VP-tree-based neighbor query against a preserved brute-force reimplement — correctness never depends on the spatial index, only performance does.

Scala-specific gotcha writing specs against this file: give any `Seq[Simplex[_]]` an explicit type ascription (`val xs: IndexedSeq[Simplex[Int]] = ...`) before calling `.forall`/similar on it — in a `specs2` spec mixing the `ScalaCheck` trait, an un-ascribed `.forall` can resolve to a `specs2` extension instead of the standard-library one, breaking type inference inside the lambda.

Opt-in parallelism

`AlphaDQPSettings.parallel` (default `false`) runs the per-vertex QP solve loop on the common `ForkJoinPool`; output is deterministic either way. Worth reaching for once a point cloud is large enough that the per-vertex solve cost dominates (roughly a few hundred points and up) — measured at 2-4x wall-clock speedup in that regime.

Honest framing worth keeping in mind while working on either backend

The paper's own benchmarks are mixed against Ripser (loses on 2 of 4 persistence examples) and against qhull-based Delaunay on some inputs. DQP's genuine value proposition is high ambient dimension (where Delaunay is infeasible), exact homology rather than persistence diagrams, and much smaller complexes than Vietoris-Rips when data sits near a low-dimensional subspace — not raw speed. See the [User's Guide](#) for how this should shape what you tell an end user.

Mapping the library: class diagrams



This page is a structural sketch to help you get oriented, not an exhaustive or automatically generated reference — field names and signatures can drift out of sync with source over time. When a diagram and the actual `.scala` file disagree, trust the file.

Typeclass hierarchy (`RingModule.scala`, `Field.scala`, `Chain.scala`)

Three concrete `OrderedCell` instances exist: `Simplex[VertexT]`, `Cube`, and a `FiniteSimplicialSet[G]`'s own generators (that last one is a per-instance given, not a global one, since its boundary depends on that particular simplicial set's own face data). There is no dual `Cocell/OrderedCocell` trait pair (an earlier version had one; removed as the wrong shape for coboundary, which is extrinsic to a cell, not intrinsic like `boundary`) — `RipserCohomologyContext/PackedRipserCohomologyContext` compute coboundaries directly against `SimplexIndexing` instead (see [Persistence engines](#)). See the [Scala 3 primer](#) for what "typeclass: type Self" and "given instance" mean concretely in this codebase's syntax.

`Chain[CellT, CoefficientT]` (`Chain.scala`)

`reduceBy/reduceByUntil` are the shared reduction primitives every persistence engine in `Homology.scala` builds on — see [Architecture](#) and [Hard-won invariants #4](#).

`Simplex[VertexT]` (`Simplex.scala`, `SimplexOps.scala`)

`SimplexOps.scala` adds a large extension block delegating most of `SortedSet`'s surface (`.size`, `.map`, `.union`, `.dropIndex`, ...) so `Simplex` "feels like" a set even though it's a zero-cost opaque wrapper at runtime — see the [primer](#).

`Streams` (`SimplexStream.scala`, `SimplexIndexing.scala`, `VietorisRips.scala`)

These are alternate stream implementations with a common output contract, not layers on top of one another — see [Architecture](#). `CubicalGridStream/ExplicitCubicalStream` produce

Cubes rather than Simplexes; `SimplicialSetStream/FilteredSimplicialSetStream` produce a `FiniteSimplicialSet[G]`'s own generator type `G`.

Persistence engines (`homology/Homology.scala`, `homology/PackedRipserCohomology.scala`)

Deliberately *not* diagrammed field-by-field here — their exact state and trust status belongs in one place. See [Persistence engines](#) for the full, current picture across `CellularHomologyContext/SimplicialHomologyContext`, `CellularPersistenceInChunksContext/PersistenceInChunksContext`, `RipserCohomologyContext`, `PackedRipserCohomologyContext`, and `CellularCohomologyContext`.

Metric spaces (`FiniteMetricSpace.scala`)

Barcode representation (`Barcode.scala`, package `org.appliedtopology.tda4j.barcode`)

`AnnotationT` in practice is always `Chain[CellT, CoefficientT]` — the representative cycle/cocycle for a bar, when an engine tracks one.